Writer Monad

Writer · Compute totals while accumulating audit log · Build with Waqar

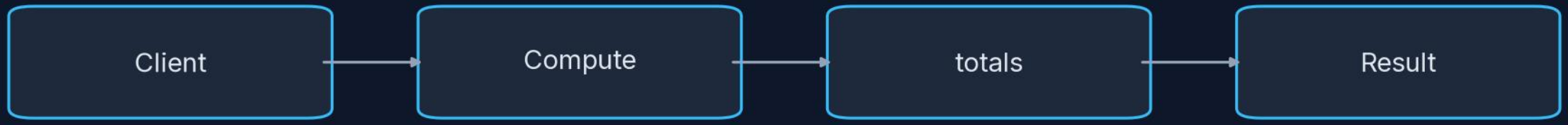
Today's Hook

- Problem: Compute totals while accumulating audit log
- Pattern: Writer — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Compute totals while accumulating audit log
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Writer: Compute totals while accumulating audit log

Real-World Scenario

- Running example: Compute totals while accumulating audit log
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.writer.problem;

import java.util.*;

public final class NaiveAudit {
  public static int total(List<Integer> xs){ return xs.stream().mapToInt(i->i).sum(); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.writer.problem;

import java.util.*;
import java.util.function.*;

record Writer<W,A>(A value, List<W> log){
  static <W,A> Writer<W,A> pure(A a){ return new Writer<>(a, List.of()); }
  <B> Writer<W,B> map(Function<A,B> f){ return new Writer<>(f.apply(value), log); }
  <B> Writer<W,B> flatMap(Function<A, Writer<W,B>> f){
    var next=f.apply(value); List<W> merged=new ArrayList<>(log); merged.addAll(next.log); return new Writer<>(next.value, merged);
  }
  static <W,A> Writer<W,A> tell(W w, A a){ return new Writer<>(a, List.of(w)); }
}
final class Audited {
  static Writer<String,Integer> addLine(int cents){ return Writer.tell("line="+cents, cents); }
  static Writer<String,Integer> total(List<Integer> lines){
    Writer<String,Integer> acc=Writer.pure(0);
    for(int c: lines){ acc = acc.flatMap(sum -> addLine(c).map(v -> sum+v)); }
    return acc;
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.writer.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Writer: totals with audit log ===");
    System.out.println("--- naive ---"); System.out.println(NaiveAudit.total(List.of(10,20)));
System.out.println("--- writer ---");
var w=Audited.total(List.of(10,20,30));
System.out.println("value="+w.value()); System.out.println("log="+w.log());
  }
}
```

Walkthrough Output

```
=== Writer: totals with audit log ===
--- naive ---
30
--- writer ---
value=60
log=[line=10, line=20, line=30]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Writer.

Related Patterns

- Contrast Writer with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Writer → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.